

PRELIM MINI PROJECT EXAM: AI-POWERED HOME VIRTUAL ASSISTANT

APEX HOME AUTOMATIONS — ON-PREMISE AI VIRTUAL ASSISTANT (PROJECT JARVIS)

Instructor: Prof. Rob Malitao	Due Date: August 21, 2026	Environment: 100% Offline Desktop (Zero-Hardware)
Student Engineers:	JOHN MIKO SARSALIJO (Lead GUI & Simulator Architect)	CHRISTIAN EZEKIEL CARVAJAL (Lead AI & Systems Architect)

1. Business Scenario & Executive Solution Overview

Company Profile & Challenge: Apex Home Automations requires a next-generation desktop virtual assistant addressing consumer privacy concerns with cloud-connected devices. The solution is **Project JARVIS**, a 100% offline, on-premise smart home assistant operating with zero cloud telemetry and zero external API dependencies.

System Pipeline: Captures voice via laptop microphone, extracts structured actions using a local Ollama LLM (Qwen 2.5:1.5b), synchronizes an interactive Tkinter graphical dashboard, and generates offline auditory TTS confirmations.

2. System Architecture & End-to-End Voice Pipeline

Stage 1: Voice Input (STT)	Stage 2: Local AI Brain (LLM)	Stage 3: GUI & Audio Output (TTS)
• 16kHz float32 audio capture • Silero VAD silence cutting (~192ms) • Pre-roll ring buffer (no syllable loss) • Faster-Whisper / SpeechRecognition • Wake-word gating ('Hey Jarvis')	• Standby <-> Command State Machine • Local Ollama Daemon (Qwen 2.5:1.5b) • Ambiguous natural language reasoning • Strict Pydantic v2 JSON validation • Audit log: assistant_execution.log	• Virtual Device State Machine • Dynamic Tkinter HUD real-time sync • Lights, Thermostat, Lock, Fan, Alarm • Offline pyttsx3 + Edge British voice • Emergency HALT speech override

3. Technical Stack & Modular OOP Source Code Compliance (/src)

Mandated Module	Implementation in Project JARVIS	Status
src/main.py	Two-Turn State Machine entry point, GUI event loop coordinator & ISO logging.	100% Match
src/voice_pipeline.py	Audio capture, Silero VAD, Faster-Whisper STT, and async TTS speech queue.	100% Match
src/ai_engine.py	Ollama Qwen 2.5 intent parser, strict Pydantic v2 schemas, zero regex triggers.	100% Match
src/home_simulator.py	OOP virtual device hierarchy (Light, Thermostat, Lock, Fan) & Tkinter GUI.	100% Match

4. Voice Command Execution & GUI State Walkthrough

Demonstrating end-to-end execution across three distinct voice interactions with before/after state transitions, validated Pydantic JSON schemas, and spoken auditory feedback:

Command 1 (Ambiguous Intent): "It's getting dark in here and I'm freezing"

Before State (Initial)	After State (GUI Update)	Validated JSON Action Payload
Living Room Light: OFF (0%) Thermostat: 21.5°C (AUTO)	Living Room Light: ON (100% Warm White) Thermostat: 24.0°C (HEAT Mode)	{ "spoken_response": "I have turned on the living room light and set the thermostat to 24 degrees.", "actions": [{"domain": "smart_home", "target": "living_room_light", "action": "turn_on"}, {"domain": "smart_home", "target": "thermostat", "action": "set_temperature", "value": 24.0}] }

Auditory Feedback: "I have turned on the living room light and set the thermostat to 24 degrees."

Command 2 (Direct Multi-Device): "Hey Sophia, set the living room thermostat to 22 degrees and turn off the kitchen lights"

Before State (Initial)	After State (GUI Update)	Validated JSON Action Payload
Kitchen Light: ON (100%) Thermostat: 24.0°C (HEAT)	Kitchen Light: OFF (0%) Thermostat: 22.0°C (AUTO Mode)	{ "spoken_response": "Living room thermostat set to 22 degrees, and kitchen lights are now off.", "actions": [{"domain": "smart_home", "target": "thermostat", "action": "set_temperature", "value": 22.0}, {"domain": "smart_home", "target": "kitchen_light", "action": "turn_off"}] }

Auditory Feedback: "Living room thermostat set to 22 degrees, and kitchen lights are now off."

Command 3 (Compound Night Routine): "Goodnight Jarvis, I am going to bed now"

Before State (Initial)	After State (GUI Update)	Validated JSON Action Payload
Bedroom Light: ON Living Light: ON Front Door Lock: UNLOCKED	Bedroom Light: OFF Living Light: OFF Front Door Lock: LOCKED	{ "spoken_response": "Goodnight, sir. All lights are powered down and the perimeter is secured.", "actions": [{"domain": "smart_home", "target": "bedroom_light", "action": "turn_off"}, {"domain": "smart_home", "target": "living_room_light", "action": "turn_off"}, {"domain": "smart_home", "target": "front_door_lock", "action": "lock"}] }

Auditory Feedback: "Goodnight, sir. All lights are powered down and the perimeter is secured."

5. AI Intent Extraction & JSON Parsing Benchmark Evaluation

Benchmark Dimension	Qwen 2.5 (1.5B Baseline)	Fine-Tuned (jarvis-trained)	Delta Improvement
JSON Validity Rate	100.0% (15/15)	100.0% (15/15)	100% Valid (Pydantic v2)
Intent Extraction Accuracy	93.3% (14/15)	100.0% (15/15)	+6.7% (Flawless Routing)
Chit-Chat False Activation	33.3% False Triggers	0.0% False Triggers	Zero False Activations
Average Inference Latency	1,210 ms (1.21 s)	480 ms (0.48 s)	-60.3% Latency Reduction

6. Peak Hardware Telemetry & Real-Time Resource Profile

Hardware resource utilization and end-to-end latency were recorded under live microphone stream conditions:

Pipeline Subsystem	Recorded Latency / Footprint	Exam Target	Status
Audio Capture & Silero VAD Slicing	190 ms – 320 ms	< 500 ms	PASSED
STT Transcription (Faster-Whisper)	280 ms – 450 ms	< 1000 ms	PASSED
Local Ollama Qwen 2.5 Inference	350 ms – 580 ms	< 1500 ms	PASSED
Pydantic v2 Schema Validation	0.8 ms – 1.6 ms	< 50 ms	PASSED
Tkinter Dashboard Visual State Sync	8 ms – 20 ms	< 100 ms	PASSED
TTS Auditory Synthesis Launch	35 ms – 75 ms	< 200 ms	PASSED
Total End-to-End Response Latency	0.95 s – 1.45 s	< 2.0 s – 3.0 s	OUTSTANDING
Peak Process CPU Utilization	11.8% – 14.2% (Multi-threaded)	< 50.0%	OPTIMAL
Peak Process RAM Working Set	280 MB (App) + 1.2 GB (Ollama)	< 4.0 GB	OPTIMAL

7. Pair-Programming Task Division Matrix

Student Engineer	Assigned Subsystems & Technical Responsibilities	Effort
JOHN MIKO SARSALIJO <i>Lead GUI & Simulator Architect</i> <i>Junior AI Systems Engineer</i>	- Designed & implemented OOP virtual device state machine in src/home_simulator.py. - Constructed real-time Stark Cyberpunk Tkinter GUI with live device cards and telemetry. - Engineered system resource telemetry monitoring (CPU, RAM, latency) and GUI dispatch in src/main.py. - Implemented structured ISO 8601 logging engine in assistant_execution.log.	50%
CHRISTIAN EZEKIEL CARVAJAL <i>Lead AI & Systems Architect</i> <i>Junior AI Systems Engineer</i>	- Configured local Ollama inference engine and fine-tuned Qwen model (jarvis-trained-model). - Built strict Pydantic v2 schema validation (AssistantIntentResponse, DeviceAction) in src/ai_engine.py. - Implemented Two-Turn State Machine & acoustic wake gating ('Hey Jarvis') in src/voice_pipeline.py. - Developed hybrid TTS engine with emergency HALT override and automated benchmark harness.	50%

8. Assessment Rubric 100-Point Compliance Summary

Criteria	Weight	Target Standard (Outstanding: 90–100%)	Achieved in JARVIS
Voice & STT Pipeline	25%	Seamless audio capture, STT, and TTS with <2s latency.	Faster-Whisper + Silero VAD + pyttsx3 (<1.45s).
AI Intent & JSON	30%	Qwen extracts intent from ambiguous prompts; 100% valid JSON.	Local Ollama Qwen 2.5 + Pydantic v2 schemas.
Smart-Home GUI	20%	Dynamic UI reflecting real-time state changes on AI output.	Tkinter dynamic cards (lights, locks, thermostat).
Code Architecture	15%	Modular OOP design, PEP-8 comments, robust error handling.	Clean /src structure, zero hardcoded paths.
Report & Live Demo	10%	Complete PDF report, task matrix, and flawless live demo.	3-page report + automated test suite.